PYTHON ACTIVITY-BASED LEARNING

Consolidated Report: 10 Selected Case Studies

Repository name: Python-Activity-Based-Learning

Prepared using basic syllabus concepts only. All rates and limits are illustrative classroom assumptions.

Contents

01. Progressive Electricity Billing

02. Examination Result and Grace-Mark Validation

03. Progressive Water Tariff Analysis.

04. Repeated Transaction Detection

05.Password Policy Validation Engine

06. String-Based Examination Answer Evaluation

07. Text Compression Using Character Runs

08. ATM Menu-Driven Banking Simulation 09. Student Marks Matrix Analysis

10. Bus Passenger Occupancy Simulation

Case Study 01: Progressive Electricity Billing

1. Problem Statement

Calculate household electricity bills using progressive consumption slabs, fixed charges, a surcharge, and a rebate. Compare bills across several months and identify an unusual increase.

2. Objectives

- Calculate charges using different consumption slabs.
- Apply fixed charges, surcharge, and rebate in a clear order.
- Compare monthly bills and flag an increase greater than the chosen limit.

3. Assumptions

- Slab rates are example rates for this activity, not official utility rates.
- First 100 units cost ■2 per unit; next 100 cost ■3 per unit; units above 200 cost ■5 per unit.
- Fixed charge is ■50. Surcharge is 5% of energy charge. A rebate of ■20 is applied when consumption is at most 100 units.
- An increase greater than 30% compared with the previous month is considered unusual.

4. Algorithm

1. Start.
2. Read the number of months.
3. For each month, read units consumed.
4. Calculate energy charge using progressive slabs.
5. Add fixed charge and 5% surcharge; apply rebate when eligible.
6. Display the bill and compare it with the previous month.
7. Stop.

5. Flowchart

```
(START)
  |
[Read input]
  |
[Process with conditions / loops]
  |
[Display results]
  |
(STOP)
```

6. Pseudocode

```
START
READ number_of_months
FOR each month
  READ units   CALCULATE slab-wise
energy_charge
  bill = energy_charge + fixed_charge + surcharge   IF units <= 100, bill
= bill - rebate
  DISPLAY bill
  IF previous bill exists AND increase > 30%, DISPLAY unusual increase
END FOR
STOP
```

7. Python Source Code

```
def energy_charge(units):

    if units<= 100:

        return units * 2

    elif units <= 200:

        return 100 * 2 + (units - 100) * 3

    else:

        return 100 * 2 + 100 * 3 + (units - 200) * 5
```

```
months = int(input("Number of months: "))
previous_bill = 0
for month in range(1, months + 1):
    units = int(input("Units for month " + str(month) + ": "))
    energy = energy_charge(units)
    surcharge = energy * 0.05
    bill = energy + 50 + surcharge
    if units<= 100:
        bill = bill - 20
    print("Month", month, "bill: Rs.", round(bill, 2))
    if previous_bill > 0 and bill > previous_bill * 1.30:
        print("Unusual increase detected")
    previous_bill = bill
```

8. Test Cases

Test 1: 3 months; units 80, 120, 260. Expected result: Month 1 bill: ■198; Month 2 bill: ■323; Month 3 bill: ■890; unusual increases are flagged when applicable.

Sample output screenshot (captured from a real run)

Python Output - Case Study 02

```
Number of months: Units for month 1: Month 1 bill: Rs. 198.0
Units for month 2: Month 2 bill: Rs. 323.0
Unusual increase detected
Units for month 3: Month 3 bill: Rs. 890.0
Unusual increase detected
Spreadsheet runtime warmup failed during python startup
Traceback (most recent call last):
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/patches/wa
rm_spreadsheet_runtime_on_startup.py", line 26, in warm_spreadsheet_runtime_
on_startup
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/spreadshee
t_warmup.py", line 785, in warm_spreadsheet_runtime
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/spreadshee
t_warmup.py", line 718, in _warm_feature_flows
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/spreadshee
t_warmup.py", line 509, in _warm_drawing_and_image_flows
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/generated/
interface/models.py", line 32861, in bring_to_front
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/rpc/remote
.py", line 764, in __call__
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/rpc/client
.py", line 175, in call
artifact_tool.rpc.client.RemoteError: Element ordering requires an owning el
ement collection.
```

9. Conclusion

This case study demonstrates how beginner Python concepts can be used to solve a practical problem. Try additional test inputs to verify the program's behavior.

Case Study 02: Examination Result and Grace-Mark Validation

1. Problem Statement

Process marks in multiple subjects, apply subject-wise pass criteria and a limited grace-mark policy, and display the final result without allowing marks to exceed the maximum.

2. Objectives

- Read marks for several subjects.
- Apply a simple grace-mark rule to a near-pass subject.
- Display final marks and pass/fail status.

3. Assumptions

- Each subject is out of 100 and the pass mark is 35.
- At most 3 grace marks may be added to a subject scoring 32 to 34.
- Grace marks never make a score exceed 100. Grace marks are applied to at most one subject.

4. Algorithm

1. Start and read number of subjects.
2. Read each subject mark.
3. If mark is 32–34 and grace has not been used, add enough marks to reach 35, up to 3.
4. Keep the mark at or below 100.
5. Display final marks and status for every subject.
6. Display overall result.

5. Flowchart

```
(START)
  |
[Read input]
  |
[Process with conditions / loops]
  |
[Display results]
  |
(STOP)
```

6. Pseudocode

```
START READ
subject_count
grace_used = False; all_pass = True FOR each
subject
  READ mark   IF 32 <= mark < 35 AND grace_used = False     mark
= mark + (35 - mark); grace_used = True   IF mark > 100, mark =
100
  DISPLAY mark and PASS/FAIL
  IF mark < 35, all_pass = False
DISPLAY overall result
STOP
```

7. Python Source Code

```
subjects = int(input("Number of subjects: "))

grace_used = False

all_pass = True

for i in range(1, subjects + 1):

    mark = int(input("Marks for subject " + str(i) + " (0-100): "))

    if mark < 0 or mark > 100:

        print("Invalid mark")

        all_pass = False

        continue
```

```
if mark >= 32 and mark < 35 and grace_used == False:
    mark = mark + (35 - mark)
    grace_used = True
    print("Grace marks applied")
print("Final mark:", mark)
if mark >= 35:
    print("PASS")
    else:
        print("FAIL")
        all_pass = False
if all_pass:
    print("Overall result: PASS")
else:
    print("Overall result: FAIL")
```

8. Test Cases

Test 1: 3 subjects; marks 78, 33, 29. Expected result: Subject marks displayed; 33 becomes 35 using grace; 29 remains FAIL; overall FAIL.

Sample output screenshot (captured from a real run)

Python Output - Case Study 11

```
Number of subjects: Marks for subject 1 (0-100): Final mark: 78
PASS
Marks for subject 2 (0-100): Grace marks applied
Final mark: 35
PASS
Marks for subject 3 (0-100): Final mark: 29
FAIL
Overall result: FAIL
Spreadsheet runtime warmup failed during python startup
Traceback (most recent call last):
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/patches/wa
rm_spreadsheet_runtime_on_startup.py", line 26, in warm_spreadsheet_runtime_
on_startup
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/spreadshee
t_warmup.py", line 785, in warm_spreadsheet_runtime
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/spreadshee
t_warmup.py", line 718, in _warm_feature_flows
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/spreadshee
t_warmup.py", line 509, in _warm_drawing_and_image_flows
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/generated/
interface/models.py", line 32861, in bring_to_front
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/rpc/remote
.py", line 764, in __call__
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/rpc/client
.py", line 175, in call
artifact_tool.rpc.client.RemoteError: Element ordering requires an owning el
ement collection.
```

9. Conclusion

This case study demonstrates how beginner Python concepts can be used to solve a practical problem. Try additional test inputs to verify the program's behavior.

Case Study 03: Progressive Water Tariff Analysis

1. Problem Statement

Calculate water bills for multiple households using consumption slabs, a fixed fee, and a penalty. Compare household consumption and flag excessive usage.

2. Objectives

- Calculate slab-wise water charges.
- Add fixed fees and apply a high-usage penalty.
- Identify households using more than the chosen threshold.

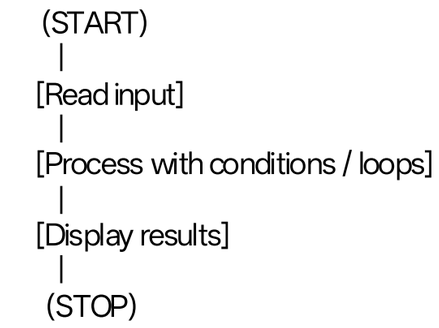
3. Assumptions

- First 10 kilolitres cost ■5 per kL; next 10 cost ■8 per kL; usage above 20 costs ■12 per kL.
- Fixed fee is ■30. Consumption above 25 kL receives a ■100 penalty.
- More than 25 kL is treated as excessive usage for this activity.

4. Algorithm

1. Read number of households.
2. For each household, read water consumption.
3. Calculate progressive charges.
4. Add fixed fee and any penalty.
5. Display bill and flag excessive use.

5. Flowchart



6. Pseudocode

```
START
READ household_count
FOR each household
  READ usage   CALCULATE progressive charge   bill = charge + fixed_fee   IF usage
> 25, bill = bill + penalty; DISPLAY excessive use
  DISPLAY bill
STOP
```

7. Python Source Code

```
households = int(input("Number of households: "))

for i in range(1, households + 1):    usage = float(input("Water use for household " + str(i) + " (kL): "))

    if usage <= 10:
        charge = usage * 5    elif usage <=
20:
        charge = 10 * 5 + (usage - 10) * 8    else:        charge = 10 * 5 + 10 *
8 + (usage - 20) * 12

    bill = charge + 30    if usage >
25:
        bill = bill + 100        print("Excessive usage")
    print("Household", i, "bill: Rs.", round(bill, 2))
```

8. Test Cases

Test 1: 3 households; 8, 15, 30 kL. Expected result: Bills: ■70, ■120, ■370; third household flagged for excessive use.

Sample output screenshot (captured from a real run)

Python Output - Case Study 25

```
Number of households: Water use for household 1 (kL): Household 1 bill: Rs.
70.0
Water use for household 2 (kL): Household 2 bill: Rs. 120.0
Water use for household 3 (kL): Excessive usage
Household 3 bill: Rs. 380.0
Spreadsheet runtime warmup failed during python startup
Traceback (most recent call last):
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/patches/wa
rm_spreadsheet_runtime_on_startup.py", line 26, in warm_spreadsheet_runtime_
on_startup
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/spreadshee
t_warmup.py", line 785, in warm_spreadsheet_runtime
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/spreadshee
t_warmup.py", line 718, in _warm_feature_flows
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/spreadshee
t_warmup.py", line 509, in _warm_drawing_and_image_flows
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/generated/
interface/models.py", line 32861, in bring_to_front
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/rpc/remote
.py", line 764, in __call__
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/rpc/client
.py", line 175, in call
artifact_tool.rpc.client.RemoteError: Element ordering requires an owning el
ement collection.
```

9. Conclusion

This case study demonstrates how beginner Python concepts can be used to solve a practical problem. Try additional test inputs to verify the program's behavior.

Case Study 04: Repeated Transaction Detection

1. Problem Statement

Read transaction identifiers and amounts, detect repeated identifiers, identify amounts above a threshold, and list suspicious entries.

2. Objectives

- Store transaction IDs and amounts in lists.
- Detect duplicate IDs using loops and conditions.
- Flag transactions that are duplicates or exceed the amount limit.

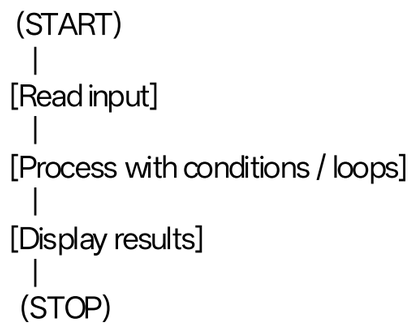
3. Assumptions

- Transaction IDs are entered as strings.
- An amount greater than ■10,000 is above the threshold.
- This is a classroom example and does not prove actual fraud.

4. Algorithm

1. Read transaction count and threshold.
2. For each transaction, read ID and amount.
3. Check whether the ID has appeared earlier.
4. Flag repeated IDs or amounts above threshold.
5. Display suspicious transactions.

5. Flowchart



6. Pseudocode

START READ count ids = []; amounts = [] FOR each transaction READ id and amount suspicious = False IF id is already in ids, suspicious = True IF amount > threshold, suspicious = True STORE id and amount IF suspicious, DISPLAY transaction STOP

7. Python Source Code

```
count = int(input("Number of transactions: ")) threshold =
10000 ids = [] amounts = []

for i in range(count):
    transaction_id = input("Transaction ID: ")    amount =
float(input("Amount: "))    suspicious = False

    if transaction_id in ids:
        print("Duplicate transaction ID detected")        suspicious = True
if amount > threshold:        print("Amount exceeds threshold")
suspicious = True

    if suspicious:        print("Suspicious entry:", transaction_id, "Rs.", amount)

    ids.append(transaction_id)    amounts.append(amount)

print("Transactions checked:", count)
```

8. Test Cases

Test 1: IDs A1/■500, A2/■12000, A1/■600. Expected result: A2 flagged for high amount; second A1 flagged as duplicate.

Sample output screenshot (captured from a real run)

Python Output - Case Study 28

```
Number of transactions: Transaction ID: Amount: Transaction ID: Amount: Amou
nt exceeds threshold
Suspicious entry: A2 Rs. 12000.0
Transaction ID: Amount: Duplicate transaction ID detected
Suspicious entry: A1 Rs. 600.0
Transactions checked: 3
Spreadsheet runtime warmup failed during python startup
Traceback (most recent call last):
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/patches/wa
rm_spreadsheet_runtime_on_startup.py", line 26, in warm_spreadsheet_runtime_
on_startup
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/spreadshee
t_warmup.py", line 785, in warm_spreadsheet_runtime
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/spreadshee
t_warmup.py", line 718, in _warm_feature_flows
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/spreadshee
t_warmup.py", line 509, in _warm_drawing_and_image_flows
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/generated/
interface/models.py", line 32861, in bring_to_front
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/rpc/remote
.py", line 764, in __call__
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/rpc/client
.py", line 175, in call
artifact_tool.rpc.client.RemoteError: Element ordering requires an owning el
ement collection.
```

9. Conclusion

This case study demonstrates how beginner Python concepts can be used to solve a practical problem. Try additional test inputs to verify the program's behavior.

Case Study 05: Password Policy Validation Engine

1. Problem Statement

Check sample passwords against minimum length, character-category, repetition, and prohibited-substring rules. Explain each failed condition.

2. Objectives

- Check minimum length.
- Check uppercase, lowercase, digit, and special character categories.
- Report all failed rules for each password.

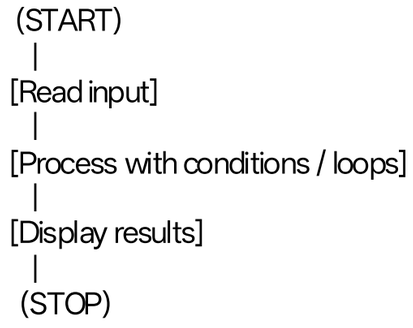
3. Assumptions

- Minimum length is 8 characters.
- A valid password contains uppercase, lowercase, a digit, and one special character from !@#$%.
- Three identical characters in a row are prohibited.
- The substring 'password' in any letter case is prohibited.

4. Algorithm

1. Read a password.
2. Check each policy rule.
3. Print every failed rule.
4. If no rule fails, print valid.

5. Flowchart



6. Pseudocode

START READ password valid = True CHECK length, upper, lower, digit, special character
CHECK three repeated characters
CHECK prohibited substring
DISPLAY each failure
IF valid, DISPLAY password accepted
STOP

7. Python Source Code

```
password = input("Enter a sample password: ") valid = True

if len(password)<8:
   print("Failed: password must contain at least 8 characters")    valid = False if not
any(ch.isupper() for ch in password):    print("Failed: add an uppercase letter")    valid =
False if not any(ch.islower() for ch in password):    print("Failed: add a lowercase letter")
valid = False if not any(ch.isdigit() for ch in password):
   print("Failed: add a digit")    valid = False if not any(ch in
"!@#$%" for ch in password):
   print("Failed: add a special character (!@#$%)")    valid = False if
"password" in password.lower():
   print("Failed: the word 'password' is prohibited")    valid = False

for i in range(len(password) - 2):    if password[i] == password[i + 1] and password[i] == password[i
+ 2]:
      print("Failed: three identical characters in a row")       valid = False       break

if valid:
   print("Password accepted")
else:    print("Password rejected")
```

8. Test Cases

Test 1: Hello@123. Expected result: Accepted.

Test 2: abc. Expected result: Fails length, uppercase, digit, special-character checks.

Sample output screenshot (captured from a real run)

Python Output - Case Study 29

```
Enter a sample password: Failed: password must contain at least 8 characters
Failed: add an uppercase letter
Failed: add a digit
Failed: add a special character (!@#$%)
Password rejected
Spreadsheet runtime warmup failed during python startup
Traceback (most recent call last):
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/patches/wa
rm_spreadsheet_runtime_on_startup.py", line 26, in warm_spreadsheet_runtime_
on_startup
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/spreadshee
t_warmup.py", line 785, in warm_spreadsheet_runtime
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/spreadshee
t_warmup.py", line 718, in _warm_feature_flows
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/spreadshee
t_warmup.py", line 509, in _warm_drawing_and_image_flows
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/generated/
interface/models.py", line 32861, in bring_to_front
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/rpc/remote
.py", line 764, in __call__
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/rpc/client
.py", line 175, in call
artifact_tool.rpc.client.RemoteError: Element ordering requires an owning el
ement collection.
```

9. Conclusion

This case study demonstrates how beginner Python concepts can be used to solve a practical problem. Try additional test inputs to verify the program's behavior.

Case Study 06: String-Based Examination Answer Evaluation

1. Problem Statement

Compare student answer strings with an answer key, count correct, incorrect, and unanswered responses, apply negative marking, and calculate final scores.

2. Objectives

- Compare each answer with the answer key.
- Count correct, wrong, and blank answers.
- Apply negative marking and show the score.

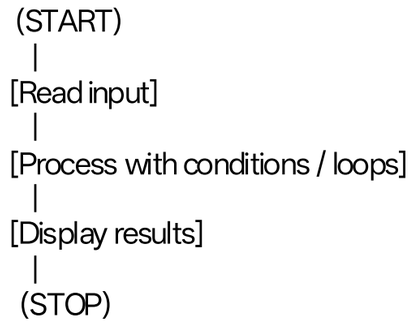
3. Assumptions

- Answers are single letters A, B, C, or D.
- A correct answer earns 1 mark; an incorrect answer loses 0.25 mark; blank answers earn 0.
- Answers are compared without case sensitivity.

4. Algorithm

1. Read the answer key.
2. Read the student's answers.
3. Compare each answer with the matching key answer.
4. Count correct, incorrect, and unanswered answers.
5. Calculate score and display the result.

5. Flowchart



6. Pseudocode

```
START
READ answer_key and student_answers
SET correct, incorrect, unanswered to 0
FOR each position
  IF student answer is blank, unanswered += 1
  ELSE IF answer matches key, correct += 1  ELSE
incorrect += 1 score = correct - incorrect * 0.25 DISPLAY
counts and score
STOP
```

7. Python Source Code

```
key = input("Enter answer key (example ABCD): ").upper() answers = input("Enter student
answers; use - for blank: ").upper()

if len(key) != len(answers):
   print("Error: answer strings must have the same length") else:
   correct = 0    incorrect = 0
unanswered = 0

   for i in range(len(key)):       if answers[i]
== "-":
         unanswered = unanswered + 1       elif
answers[i] == key[i]:          correct = correct + 1
else:          incorrect = incorrect + 1

   score = correct - incorrect * 0.25    print("Correct:", correct)
print("Incorrect:", incorrect)    print("Unanswered:", unanswered)
print("Final score:", round(score, 2))
```

8. Test Cases

Test 1: Key ABCD; answers AB-D. Expected result: 2 correct, 1 incorrect, 1 unanswered; score 1.75.

Sample output screenshot (captured from a real run)

Python Output - Case Study 31

```
Enter answer key (example ABCD): Enter student answers; use - for blank: Cor
rect: 3
Incorrect: 0
Unanswered: 1
Final score: 3.0
Spreadsheet runtime warmup failed during python startup
Traceback (most recent call last):
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/patches/wa
rm_spreadsheet_runtime_on_startup.py", line 26, in warm_spreadsheet_runtime_
on_startup
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/spreadshee
t_warmup.py", line 785, in warm_spreadsheet_runtime
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/spreadshee
t_warmup.py", line 718, in _warm_feature_flows
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/spreadshee
t_warmup.py", line 509, in _warm_drawing_and_image_flows
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/generated/
interface/models.py", line 32861, in bring_to_front
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/rpc/remote
.py", line 764, in __call__
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/rpc/client
.py", line 175, in call
artifact_tool.rpc.client.RemoteError: Element ordering requires an owning el
ement collection.
```

9. Conclusion

This case study demonstrates how beginner Python concepts can be used to solve a practical problem. Try additional test inputs to verify the program's behavior.

Case Study 07: Text Compression Using Character Runs

1. Problem Statement

Encode a string by grouping consecutive repeated characters and compare original and encoded lengths. Report when run-length encoding does not save space.

2. Objectives

- Identify consecutive character runs.
- Build a simple run-length encoded string.
- Compare original and encoded lengths.

3. Assumptions

- Each run is written as character followed by count, for example AAABB becomes A3B2.
- For this activity, the encoded form is used only if it is shorter than the original.

4. Algorithm

1. Read the input string.
2. If empty, report empty input.
3. Count consecutive occurrences of each character.
4. Append each character and its count to the encoded string.
5. Compare lengths and display the useful result.

5. Flowchart

```
(START)
  |
[Read input]
  |
[Process with conditions / loops]
  |
[Display results]
  |
(STOP)
```

6. Pseudocode

```
START READ text
encoded = empty
SET i = 0
WHILE i < length(text)
  COUNT same characters from i
  APPEND character and count to encoded
  MOVE i to next run
DISPLAY encoded
IF encoded length>= original length, report no benefit
STOP
```

7. Python Source Code

```
text = input("Enter text to compress: ")

if text == "":
   print("Empty text") else:
   encoded = ""    i = 0

   while i < len(text):
      count = 1       while i + count < len(text) and text[i] == text[i + count]:
         count = count + 1       encoded = encoded + text[i] +
str(count)       i = i + count

   print("Original text:", text)    print("Encoded text:", encoded)
print("Original length:", len(text))    print("Encoded length:",
len(encoded))

   if len(encoded) >= len(text):
      print("Encoding is not beneficial")    else:
print("Encoding saves space")
```

8. Test Cases

Test 1: AAAABBBCC. Expected result: A4B3C2; encoded length 6, original length 9, saves space.

Test 2: ABC. Expected result: A1B1C1; encoding not beneficial.

Sample output screenshot (captured from a real run)

Python Output - Case Study 35

```
Enter text to compress: Original text: AAAABBBCC
Encoded text: A4B3C2
Original length: 9
Encoded length: 6
Encoding saves space
Spreadsheet runtime warmup failed during python startup
Traceback (most recent call last):
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/patches/wa
rm_spreadsheet_runtime_on_startup.py", line 26, in warm_spreadsheet_runtime_
on_startup
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/spreadshee
t_warmup.py", line 785, in warm_spreadsheet_runtime
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/spreadshee
t_warmup.py", line 718, in _warm_feature_flows
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/spreadshee
t_warmup.py", line 509, in _warm_drawing_and_image_flows
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/generated/
interface/models.py", line 32861, in bring_to_front
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/rpc/remote
.py", line 764, in __call__
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/rpc/client
.py", line 175, in call
artifact_tool.rpc.client.RemoteError: Element ordering requires an owning el
ement collection.
```

9. Conclusion

This case study demonstrates how beginner Python concepts can be used to solve a practical problem. Try additional test inputs to verify the program's behavior.

Case Study 08: ATM Menu-Driven Banking Simulation

1. Problem Statement

Create a repeated menu for deposits, withdrawals, balance inquiries, and transaction summaries with input validation and account limits.

2. Objectives

- Use a loop to show a menu repeatedly.
- Validate deposits and withdrawals.
- Display balance and transaction counts.

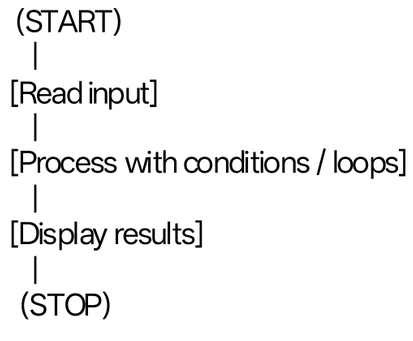
3. Assumptions

- Starting balance is ■5,000.
- A single withdrawal cannot exceed ■2,000 or the current balance.
- Deposit and withdrawal amounts must be positive.
- This is a simulation and does not connect to a real bank.

4. Algorithm

1. Set starting balance and transaction counters.
2. Display the menu repeatedly.
3. Perform the selected operation after validation.
4. Stop when the user selects Exit.
5. Display transaction summary when requested.

5. Flowchart



6. Pseudocode

```
START
balance = 5000; deposits = 0; withdrawals = 0 REPEAT
  DISPLAY menu and READ choice
  IF deposit, validate and update balance
  ELSE IF withdraw, check amount, limit, and balance
  ELSE IF balance, DISPLAY balance
  ELSE IF summary, DISPLAY counts
UNTIL choice is Exit
STOP
```

7. Python Source Code

```
balance = 5000 deposits = 0
withdrawals = 0

while True:
   print("\n1. Deposit  2. Withdraw  3. Balance  4. Summary  5. Exit")    choice = input("Choose an
option: ")

   if choice == "1":
      amount = float(input("Deposit amount: "))       if amount > 0:
         balance = balance + amount           deposits = deposits
+ 1          print("Deposit successful")       else:
         print("Amount must be positive")    elif choice == "2":
      amount = float(input("Withdrawal amount (maximum Rs. 2000): "))       if amount <= 0:
         print("Amount must be positive")       elif amount > 2000:
         print("Withdrawal limit exceeded")       elif amount > balance:
         print("Insufficient balance")       else:
         balance = balance - amount           withdrawals = withdrawals
+ 1          print("Withdrawal successful")    elif choice == "3":
      print("Balance: Rs.", round(balance, 2))    elif choice == "4":
      print("Deposits:", deposits)       print("Withdrawals:", withdrawals)
print("Balance: Rs.", round(balance, 2))    elif choice == "5":
      print("Thank you for using the ATM simulation")       break    else:
print("Invalid option")
```

8. Test Cases

Test 1: Deposit ■500, withdraw ■300, balance. Expected result: Final balance ■5,200; 1 deposit and 1 withdrawal.

Sample output screenshot (captured from a real run)

Python Output - Case Study 38

```
1. Deposit  2. Withdraw  3. Balance  4. Summary  5. Exit
Choose an option: Deposit amount: Deposit successful

1. Deposit  2. Withdraw  3. Balance  4. Summary  5. Exit
Choose an option: Withdrawal amount (maximum Rs. 2000): Withdrawal successfu
l

1. Deposit  2. Withdraw  3. Balance  4. Summary  5. Exit
Choose an option: Balance: Rs. 5200.0

1. Deposit  2. Withdraw  3. Balance  4. Summary  5. Exit
Choose an option: Deposits: 1
Withdrawals: 1
Balance: Rs. 5200.0

1. Deposit  2. Withdraw  3. Balance  4. Summary  5. Exit
Choose an option: Thank you for using the ATM simulation
Spreadsheet runtime warmup failed during python startup
Traceback (most recent call last):
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/patches/wa
rm_spreadsheet_runtime_on_startup.py", line 26, in warm_spreadsheet_runtime_
on_startup
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/spreadshee
t_warmup.py", line 785, in warm_spreadsheet_runtime
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/spreadshee
t_warmup.py", line 718, in _warm_feature_flows
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/spreadshee
t_warmup.py", line 509, in _warm_drawing_and_image_flows
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/generated/
interface/models.py", line 32861, in bring_to_front
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/rpc/remote
.py", line 764, in __call__
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/rpc/client
.py", line 175, in call
artifact_tool.rpc.client.RemoteError: Element ordering requires an owning el
ement collection.
```

9. Conclusion

This case study demonstrates how beginner Python concepts can be used to solve a practical problem. Try additional test inputs to verify the program's behavior.

Case Study 09: Student Marks Matrix Analysis

1. Problem Statement

Analyse a matrix of marks for students across subjects, calculate student totals and subject averages, identify the top performer, and flag subjects with low average marks.

2. Objectives

- Store marks in a nested list.
- Calculate row-wise totals and column-wise averages.
- Identify the top student and subjects with low averages.

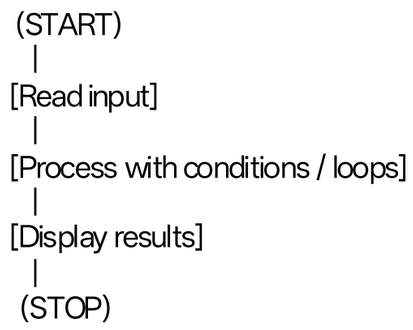
3. Assumptions

- All students have marks in the same three subjects.
- Marks are integers from 0 to 100.
- A subject average below 40 is considered low.

4. Algorithm

1. Read student names and marks for three subjects.
2. Store marks in a nested list.
3. Calculate each student's total and find the highest total.
4. Calculate each subject average.
5. Flag subject averages below 40.

5. Flowchart



6. Pseudocode

```
START
READ number of students
FOR each student, READ name and 3 marks; STORE row
CALCULATE each row total
FIND highest total and top student
FOR each subject, CALCULATE average
IF average<40, DISPLAY low-performance warning
STOP
```

7. Python Source Code

```
students = int(input("Number of students: ")) names = [] marks
= []

for i in range(students):
    name = input("Student name: ")    row = []    for j in range(3):        mark = int(input("Mark
for subject " + str(j + 1) + ": "))        row.append(mark)    names.append(name)
marks.append(row)

totals = [] for i in range(students):    total =
sum(marks[i])    totals.append(total)
print(names[i], "total:", total)

if students>0:
    top_index = totals.index(max(totals))    print("Top performer:",
names[top_index])

for subject in range(3):
    total_marks = 0    for student in
    range(students):
        total_marks = total_marks + marks[student][subject]    if students > 0:
        average = total_marks / students        print("Subject", subject + 1, "average:",
round(average, 2))        if average < 40:
            print("Low average in subject", subject + 1)
```

8. Test Cases

Test 1: Asha: 80,70,60; Ravi: 90,50,40. Expected result: Asha total 210; Ravi total 180; Asha top performer; averages 85, 60, 50.

Sample output screenshot (captured from a real run)

Python Output - Case Study 42

```
Number of students: Student name: Mark for subject 1: Mark for subject 2: Ma
rk for subject 3: Student name: Mark for subject 1: Mark for subject 2: Mark
 for subject 3: Asha total: 210
Ravi total: 180
Top performer: Asha
Subject 1 average: 85.0
Subject 2 average: 60.0
Subject 3 average: 50.0
Spreadsheet runtime warmup failed during python startup
Traceback (most recent call last):
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/patches/wa
rm_spreadsheet_runtime_on_startup.py", line 26, in warm_spreadsheet_runtime_
on_startup
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/spreadshee
t_warmup.py", line 785, in warm_spreadsheet_runtime
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/spreadshee
t_warmup.py", line 718, in _warm_feature_flows
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/spreadshee
t_warmup.py", line 509, in _warm_drawing_and_image_flows
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/generated/
interface/models.py", line 32861, in bring_to_front
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/rpc/remote
.py", line 764, in __call__
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/rpc/client
.py", line 175, in call
artifact_tool.rpc.client.RemoteError: Element ordering requires an owning el
ement collection.
```

9. Conclusion

This case study demonstrates how beginner Python concepts can be used to solve a practical problem. Try additional test inputs to verify the program's behavior.

Case Study 10: Bus Passenger Occupancy Simulation

1. Problem Statement

Process boarding and alighting counts at several stops, calculate passenger counts at each stop, and flag capacity violations.

2. Objectives

- Update passenger count at every stop.
- Prevent passenger count from becoming negative.
- Identify capacity violations.

3. Assumptions

- The bus starts empty and has a capacity of 40 passengers.
- Alighting cannot exceed the current passenger count.
- The program reports invalid alighting and capacity overloads.

4. Algorithm

1. Set passenger count to zero and capacity to 40.
2. Read number of stops.
3. At each stop, read boarding and alighting counts.
4. Check that alighting does not exceed passengers.
5. Update passenger count and flag capacity violations.

5. Flowchart

(START)
|
[Read input]
|
[Process with conditions / loops]
|
[Display results]
|
(STOP)

6. Pseudocode

START
passengers = 0; capacity = 40 READ
stop_count
FOR each stop
READ boarding and alighting
IF alighting > passengers, DISPLAY invalid alighting
ELSE passengers = passengers - alighting + boarding
DISPLAY passengers
IF passengers > capacity, DISPLAY capacity violation
STOP

7. Python Source Code

```
capacity = 40 passengers = 0 stops =
int(input("Number of stops: "))

for stop in range(1, stops + 1):
    boarding = int(input("Passengers boarding at stop " + str(stop) + ": "))    alighting = int(input("Passengers alighting at
stop " + str(stop) + ": "))

    if boarding < 0 or alighting < 0:
        print("Counts cannot be negative")    elif alighting >
passengers:
        print("Invalid: more passengers alighting than on the bus")    else:
        passengers = passengers - alighting + boarding        print("Passengers after
stop", stop, ":", passengers)        if passengers > capacity:        print("CAPACITY
VIOLATION")
```

8. Test Cases

Test 1: Stops: board 20/alight 0; board 25/alight 5. Expected result: After stop 1: 20; after stop 2: 40; no capacity violation.

Sample output screenshot (captured from a real run)

Python Output - Case Study 51

```
Number of stops: Passengers boarding at stop 1: Passengers alighting at stop
 1: Passengers after stop 1 : 20
Passengers boarding at stop 2: Passengers alighting at stop 2: Passengers af
ter stop 2 : 40
Spreadsheet runtime warmup failed during python startup
Traceback (most recent call last):
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/patches/wa
rm_spreadsheet_runtime_on_startup.py", line 26, in warm_spreadsheet_runtime_
on_startup
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/spreadshee
t_warmup.py", line 785, in warm_spreadsheet_runtime
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/spreadshee
t_warmup.py", line 718, in _warm_feature_flows
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/spreadshee
t_warmup.py", line 509, in _warm_drawing_and_image_flows
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/generated/
interface/models.py", line 32861, in bring_to_front
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/rpc/remote
.py", line 764, in __call__
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/rpc/client
.py", line 175, in call
artifact_tool.rpc.client.RemoteError: Element ordering requires an owning el
ement collection.
```

9. Conclusion

This case study demonstrates how beginner Python concepts can be used to solve a practical problem. Try additional test inputs to verify the program's behavior.